

Experiment-14 15 16

Introduction

I ran three experiments to build a model that detects if a photo is original or recaptured. Recaptured photos are pictures taken of other pictures, like from a screen, and they can have patterns like lines or color changes. The goal was to train a computer model to spot these.

The pipeline works in these short steps:

- Collect photos from datasets and convert them to the same format (JPG).
- Split the photos into training and testing groups.
- Use a pre-trained model (ResNet50) and train it with small parts (patches) of photos to find patterns.
- Test the model using multiple parts of each photo and average the results.
- Save all results like accuracy numbers and charts.

I used the same setup for all three experiments, but with different datasets. Each run took about 5-6 hours on my computer.

Model Details and Hyperparameters

The model is based on ResNet50, a popular image recognition tool pre-trained on many photos. I added layers to make it fit my task: tell original from recaptured photos.

- Main parts: ResNet50 base + pooling layer + dropout (to avoid mistakes) + two more layers for decisions.
- Input: Photos resized to 256x256, then cut into 224x224 patches.
- Training: Random patches for learning, center patch for checking during training, 5 patches averaged for final tests.
- Optimizer: Adam (helps learn better).
- Loss: Binary cross-entropy (measures errors in yes/no decisions).
- Changes to photos: Flip sides, change brightness and contrast a bit.
- Other settings:
 - Learning rate: 0.0001
 - Batch size: 16 (group of photos processed at once)
 - Epochs: 8 (full passes through data)
 - Folds: 5 for cross-check (but some runs show 3)
 - Dropout rate: 0.4
 - Repeat data: 4 times for more practice
 - Test patches: 5 per photo

Short Procedure

1. Get photos from folders and change to JPG if needed.
2. Mix originals and recaptures from all sources.
3. Split 85% for training, 15% for testing.
4. Train model in 5 parts (folds) to check fairness.
5. Stop early if no improvement, adjust learning speed.
6. Test using multiple photo parts for better score.
7. Save numbers, tables, and files for review.

Results

I ran three experiments with different data. Here are the details for each.

Experiment 14: NTU-Roselab Dataset Only

- Data: 774 originals, 976 recaptures (total 1750).
- Train: 1486 photos. Test: 264 photos.
- Average Validation Accuracy: 0.70
- Average Test Accuracy: 0.67

Per-Fold Accuracies:

Fold	Validation Accuracy	Test Accuracy
1	0.71	0.70
2	0.72	0.69
3	0.67	0.63

Example from Fold 1 - Test Confusion Matrix:

	Predicted Original	Predicted Recaptured
Actual Original	66	51
Actual Recaptured	28	119

Classification Report (Fold 1 Test, rounded):

Class	Precision	Recall	F1-Score	Support
Original	0.70	0.56	0.63	117
Recaptured	0.70	0.81	0.75	147
Average	0.70	0.69	0.69	264

The model is better at finding recaptured photos but sometimes mistakes originals.

Experiment 15: Mobile Devices Dataset Only

- Data: 408 originals, 391 recaptures (total 799).
- Devices: Iphone-11 (30 orig/30 recap), Iphone-16 (40/27), Nothing_2a (105/103), Pixel (100/100), Poco-M3 (133/131).
- Train: 678 photos. Test: 121 photos.
- Average Validation Accuracy: 0.76
- Average Test Accuracy: 0.78

Per-Fold Accuracies:

Fold	Validation Accuracy	Test Accuracy
1	0.76	0.80
2	0.75	0.77
3	0.71	0.79
4	0.78	0.75
5	0.81	0.79

Example from Fold 1 - Test Confusion Matrix:

	Predicted Original	Predicted Recaptured
Actual Original	55	7
Actual Recaptured	17	42

Classification Report (Fold 1 Test, rounded):

Class	Precision	Recall	F1-Score	Support
Original	0.76	0.89	0.82	62
Recaptured	0.86	0.71	0.78	59
Average	0.81	0.80	0.80	121

This dataset has fewer photos but from specific phones. The model does better here, with higher accuracy.

Experiment 16: Combined Datasets (NTU-Roselab + Mobile Devices)

- Data: 1182 originals, 1367 recaptures (total 2549).
- Sources: NTU (774 orig/976 recap), Devices (408 orig/391 recap).
- Train: 2165 photos. Test: 384 photos.
- Average Validation Accuracy: 0.65
- Average Test Accuracy: 0.66

Per-Fold Accuracies:

Fold	Validation Accuracy	Test Accuracy
1	0.67	0.68
2	0.66	0.66
3	0.61	0.63

Example from Fold 1 - Test Confusion Matrix:

	Predicted Original	Predicted Recaptured
Actual Original	130	48
Actual Recaptured	74	132

Classification Report (Fold 1 Test, rounded):

Class	Precision	Recall	F1-Score	Support
Original	0.64	0.73	0.68	178
Recaptured	0.73	0.64	0.68	206
Average	0.69	0.69	0.68	384

Combining datasets gives more photos, but accuracy drops a bit. Maybe because the photos from different sources are not the same.

Comparisons

Across My Experiments

- Experiment 14 (NTU only): Avg Val 0.70, Test 0.67. Good on larger dataset, but imbalance hurts originals.
- Experiment 15 (Mobiles only): Avg Val 0.76, Test 0.78. Best performance, maybe because fewer but consistent photos from 5 phones.
- Experiment 16 (Combined): Avg Val 0.65, Test 0.66. More data, but lower accuracy. Mixing sources might confuse the model due to different photo styles.

The mobile dataset alone works best. Combining helps with size but needs better mixing methods.

With Other Research on NTU-Roselab Dataset

I looked at other studies on the NTU-Roselab dataset for recapture detection.

- One paper got 98.36% accuracy using CNN with KNN classifier. My model got 0.70, so theirs is better. They might have used different methods or more tuning.
- Another study on domain adaptation for face anti-spoofing (related to recapture) reported accuracies around 90-95%. My results are lower, maybe because I used general photos, not just faces.
- A 2021 paper using the ROSE recaptured dataset (same as NTU) achieved high performance with special features. My patch-based approach is good but can improve with their ideas.

My model is a start, but others get higher scores with custom features or more data.

Conclusion

In these experiments, I built a model to detect recaptured photos using ResNet50 and patches. Experiment 15 on mobile devices gave the best results (0.78 test accuracy), while combining datasets in 16 lowered it to 0.66. This shows more data is not always better if sources differ.

I found that patch training helps spot small patterns, but imbalance makes the model miss originals sometimes. Compared to other research, my accuracies are okay but lower than 98% in some papers—maybe I can add their methods like better classifiers.

From this, I learnt how to handle datasets, train models, and check results. It took 5-6 hours per run, teaching me patience and how to fix errors. Next, I can try more folds or new datasets to improve.